Reporte

Nombre: Equipo 6 Luis David Cigala Montes Jared Alejandro Garcia Alcocer Gabriel Sánchez Rivera	**Matrícula**: 2120993 2119041 2121131
Nombre del curso: Componentes de lógica de negocio	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 3	**Actividad**: Actividad 3, Diseño de arquitectura del sistema
Fecha: 10/15/26	
Bibliografia (Formato APA): Archivo de la actividad 3 brindada por el ingeniero	

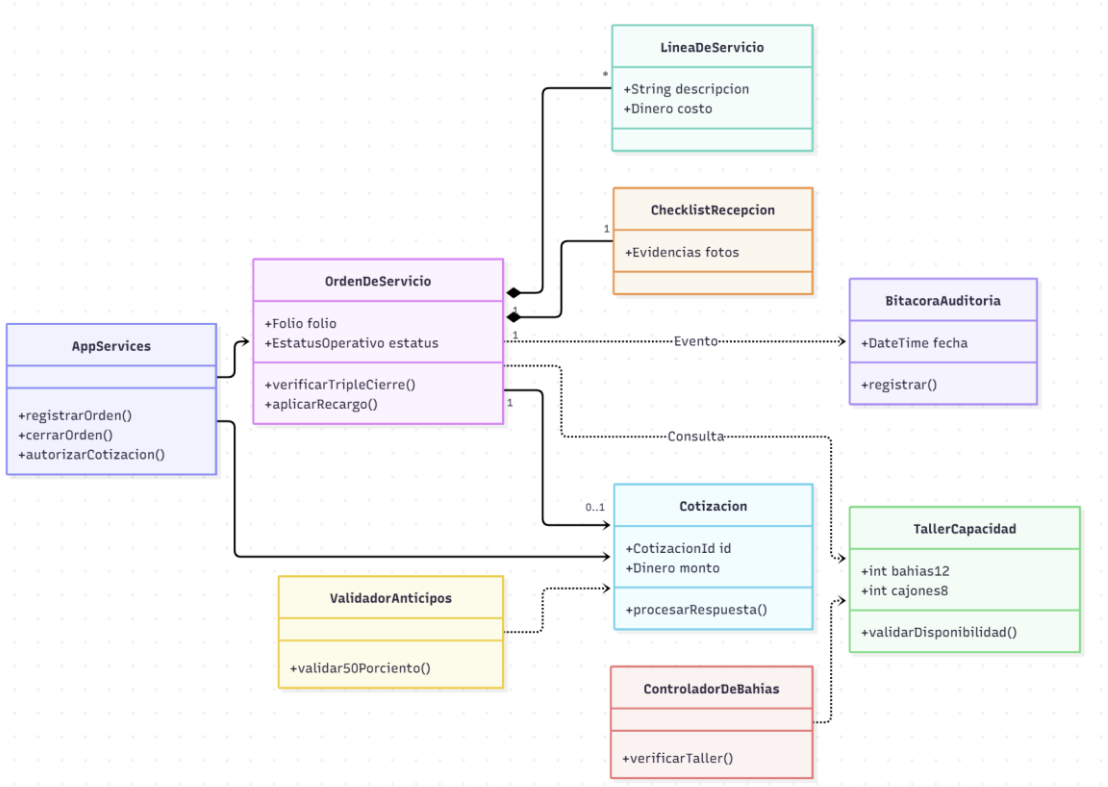
Objetivo:

Presentar por separado el diagrama de clases UML y los dos diagramas de secuencia de la propuesta de arquitectura de AutoFix Pro (sección 5 del reporte), y explicar qué muestra cada uno y cómo refleja las reglas de negocio del sistema.

Procedimiento:

1. Diagrama de clases UML

Diagrama del equipo: sección 5 del reporte



Idea general

El diagrama organiza el sistema alrededor de un aggregate raíz, OrdenDeServicio. Todo lo que debe cambiar junto con la orden (sus líneas de servicio y su checklist de recepción) vive dentro de ella; lo demás son colaboradores que se consultan o se notifican, pero no forman parte de su consistencia interna. Arriba de todo está una capa de aplicación (AppServices) que coordina los casos de uso sin contener reglas de negocio.

Clases y su responsabilidad

Clase	Atributos y operaciones	Responsabilidad
AppServices	registrarOrden(), cerrarOrden(), autorizarCotizacion()	Capa de aplicación (Service Layer). Recibe las peticiones y orquesta a las demás clases. Es el punto de entrada de los casos de uso CU-01, CU-05 y CU-02.
OrdenDeServicio	Folio folio, EstatusOperativo estatus; verificarTripleCierre(), aplicarRecargo()	Aggregate raíz. Guarda la identidad (folio) y el estatus de la orden. Verifica la condición triple de cierre (reparado, pagado y firmado) y aplica el cargo de almacenaje.
LineaDeServicio	String descripcion, Dinero costo	Cada trabajo o refacción de la orden. Sirve para acumular el monto y detectar trabajos adicionales.
ChecklistRecepcion	Evidencias fotos	Registro del estado del vehículo al recibirlo, con evidencia fotográfica.

Clase	Atributos y operaciones	Responsabilidad
Cotizacion	CotizacionId id, Dinero monto; procesarRespuesta()	Presupuesto enviado al cliente. Procesa su respuesta (autoriza o rechaza).
ValidadorAnticipos	validar50Porciento()	Domain Service. Comprueba la regla del anticipo del 50 % cuando la cotización supera $5,000.
TallerCapacidad	int bahias12, int cajones8; validarDisponibilidad()	Capacidad operativa del taller (12 bahías y 8 cajones). Responde si hay lugar para un vehículo nuevo.
ControladorDeBahias	verificarTaller()	Domain Service. Consulta TallerCapacidad para decidir si se puede aceptar el vehículo.
BitacoraAuditoria	DateTime fecha; registrar()	Registro de auditoría de los eventos importantes, como el cierre de una orden.

Relaciones

Composición (rombo negro) con LineaDeServicio y ChecklistRecepcion. Si se elimina la orden, desaparecen sus líneas y su checklist. Una orden tiene muchas líneas (*) y exactamente un checklist (1).

Asociación con Cotizacion (1 a 0..1). Una orden puede tener a lo más una cotización; si aún no se genera, no hay ninguna.

Dependencia punteada «Evento» hacia BitacoraAuditoria. OrdenDeServicio no escribe en la bitácora directamente: emite un evento y la bitácora lo registra.

Dependencia punteada «Consulta» hacia TallerCapacidad. Al registrar la orden se consulta la capacidad del taller; la orden no la posee.

Flechas sólidas desde AppServices. La capa de aplicación usa OrdenDeServicio y Cotizacion para ejecutar cada caso de uso.

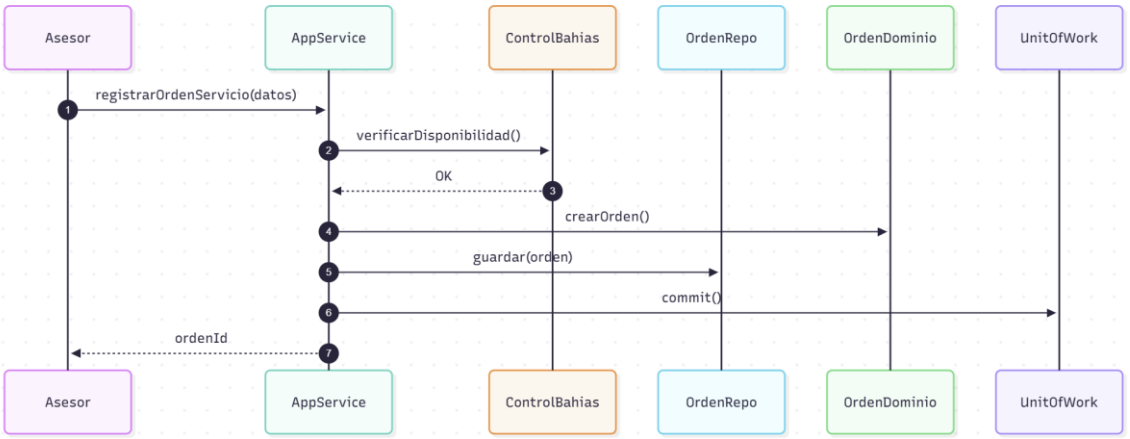
Dependencias punteadas de los Domain Services. ValidadorAnticipos hacia Cotizacion y ControladorDeBahias hacia TallerCapacidad: contienen reglas que no pertenecen a una sola entidad.

Reglas de negocio que se ven reflejadas

Regla	Dónde aparece
Capacidad del taller (US-11)	TallerCapacidad y ControladorDeBahias
Anticipo del 50 % en cotizaciones mayores a $5,000 (US-06)	ValidadorAnticipos y Cotizacion
Condición triple de cierre (US-02) y cargo de almacenaje (US-13)	verificarTripleCierre() y aplicarRecargo() en OrdenDeServicio
Bitácora de auditoría (US-19)	BitacoraAuditoria, alimentada por eventos

2. Diagrama de secuencia 1, flujo normal

Caso de uso CU-01: Recepción e ingreso de vehículo (registro de una orden de servicio)



Participantes

Participante	Función
Asesor	Persona que atiende al cliente y captura los datos.
AppService	Service Layer; coordina todo el flujo.
ControlBahias	Domain Service que verifica la capacidad del taller.
OrdenRepo	Repositorio que guarda la orden.
OrdenDominio	Aggregate OrdenDeServicio, que construye la orden aplicando sus reglas.
UnitOfWork	Confirma la transacción completa.

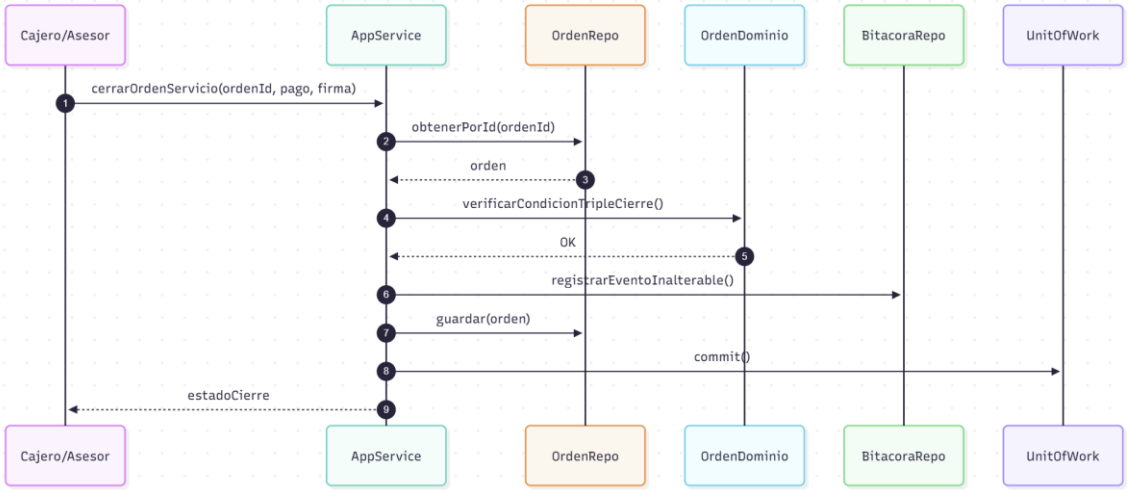
Paso a paso

Paso	Mensaje	Qué ocurre
1	registrarOrdenServicio(datos)	El asesor envía al AppService los datos del cliente y del vehículo.
2	verificarDisponibilidad()	El AppService pregunta a ControlBahias si hay capacidad operativa (US-11).
3	OK	Hay lugar disponible, por lo que el flujo continúa.
4	crearOrden()	El AppService pide al dominio crear la orden. Ahí se aplican las invariantes, como los datos obligatorios (US-15), y se genera el folio.
5	guardar(orden)	La orden creada se entrega al repositorio.
6	commit()	UnitOfWork confirma los cambios como una sola transacción.
7	ordenId	El AppService devuelve al asesor el identificador (folio) de la orden.

Lectura del diagrama: la lógica de negocio se queda en el dominio (pasos 2 a 4) y el AppService solo orquesta. La persistencia (pasos 5 y 6) ocurre al final y de forma atómica: si algo falla antes del commit, no se guarda nada.

3. Diagrama de secuencia 2, cierre de orden

Caso de uso CU-05: Cierre de orden y entrega. En el reporte se rotula «flujo con rechazo o excepción»



Participantes

Participante	Función
Cajero/Asesor	Quien registra el pago y la firma de conformidad.
AppService	Coordina el cierre.
OrdenRepo	Recupera y guarda la orden.
OrdenDominio	Aggregate que valida si la orden puede cerrarse.
BitacoraRepo	Guarda el evento de auditoría.
UnitOfWork	Confirma la transacción.

Paso a paso

Paso	Mensaje	Qué ocurre
1	cerrarOrdenServicio(ordenId, pago, firma)	El cajero o asesor solicita el cierre e indica el pago y la firma de conformidad.
2	obtenerPorId(ordenId)	El AppService pide la orden al repositorio.
3	orden	El repositorio devuelve el aggregate cargado.
4	verificarCondicionTripleCierre()	El dominio comprueba que la orden esté reparada, pagada y firmada (US-02).
5	OK	Las tres condiciones se cumplen; el cierre procede.
6	registrarEventoInalterable()	Se deja constancia en la bitácora de auditoría, que no se puede modificar (US-19).
7	guardar(orden)	Se guarda la orden con su nuevo estatus.
8	commit()	UnitOfWork confirma orden y bitácora en una sola transacción.
9	estadoCierre	Se informa al usuario el resultado del cierre.

Lectura del diagrama: la decisión de cerrar la toma el dominio, no quien llama. Orden y bitácora se guardan en la misma transacción, de modo que nunca queda una orden cerrada sin su registro de auditoría ni al revés.

Observación sobre este diagrama

Este diagrama muestra solo el camino en el que la verificación responde OK, es decir, el cierre exitoso. Aunque el rótulo habla de rechazo o excepción, no aparece la rama en la que la condición triple falla (por ejemplo, falta el pago o la firma). Para que represente un flujo de excepción conviene agregar un fragmento alt después del paso 4: si la verificación falla, el dominio rechaza el cierre, no se escribe en la bitácora, no se hace commit y el AppService devuelve el motivo del rechazo (o, si pasaron más de 15 días sin que el cliente recoja el vehículo, el estatus «en resguardo» con cargo de almacenaje, US-13). La misma observación aplica al diagrama 1, donde tampoco se ve la rama en la que ControlBahias responde que no hay capacidad (evento CapacidadSaturada).

Conclusiones

El diagrama de clases muestra la estructura: qué piezas existen y cómo se relacionan. Los diagramas de secuencia muestran el comportamiento: en qué orden colaboran esas piezas en dos casos de uso concretos. En ambos flujos el patrón es el mismo: el Service Layer recibe la petición, el dominio decide y aplica las reglas, el repositorio persiste y el Unit of Work cierra la transacción.